EXECUTE THE PROGRAM: AUDITING SEARCH AND TOOL USE IN MESH-TO-CAD RECONSTRUCTION

Anonymous authors
Paper under double-blind review

ABSTRACT

Mesh-to-CAD methods return a program, yet they are usually scored on the mesh their search reports. We audit an IoU-driven mesh-to-CAD optimizer, CADFit, together with three search extensions, by executing every final program in a fresh process and judging the solid it builds. On 60 never-inspected hard-tier parts from CADBench (benchA and benchE), the beam-search arm reports a mean mesh IoU of 0.880 but its executed programs score 0.799; 6 of its programs do not build, and they account for 0.078 of the 0.081 gap. Executed-program gains are smaller than mesh gains for the rule package and for the full ladder of extensions. Whole-part section-extrusion proposals give the largest observed executed-program increment among the extensions (+0.089 over beam search), concentrated on benchE. We then expose the optimizer's stages, including sketch extraction, extrusion and revolve fitting, and section extrusion, as tools for six open-weight LLM agents. On 20 parts, the tools raise every model's executed-program IoU over writing CadQuery alone, by 0.256 to 0.471 (all six Bonferroni-corrected intervals exclude zero). The best agent, Gemma-4-31B, scores 0.892, an observed -0.012 relative to the strongest pipeline arm on the same parts. A scripted policy over the same tools and budget scores **PENDING**, and an advantage of LLM planning over it is not established at this sample size.

1 INTRODUCTION

The product of mesh-to-CAD reconstruction is a program. A designer edits its sketches and features, and a CAD kernel turns it into a solid. Optimization-based reconstructors such as CADFit (Nehme et al., 2026) build that program by proposing extrusions and revolves fitted to slices of the target mesh, composing them with booleans, and keeping the composition with the highest volumetric IoU. In practice the score that steers and reports the search is computed on meshes: candidate solids are tessellated, combined with mesh booleans, and compared with the target. The program assembled at the end is a separate artifact. Whether it builds, whether the kernel accepts it, and whether it reproduces the reported geometry are not part of the number.

This gap matters more as search gets stronger. A search that explores more compositions has more chances to find meshes whose program cannot be replayed, and a measurement error that favours some candidates changes which compositions win. It also matters for the recent wave of LLM agents for CAD (Mallis et al., 2025; Savrasov et al., 2026), which give a language model geometric tools and report reconstruction quality. CADFather (Savrasov et al., 2026) ablates its tools but leaves open how much of an agent's result comes from the tools and how much from the model's choices among them. Answering that requires a non-LLM policy over the same tools and budget, and a metric that cannot be gamed by unexecutable output.

We study three questions on a cohort of 60 hard-tier CADBench parts (Doris et al., 2026) drawn after, and disjoint from, every part used during development:

Q1 Does mesh IoU track the IoU of the executed program (IoU_{CAD}) as the search gets stronger?

Q2 Which search extensions improve the executed program, not just the reported mesh?

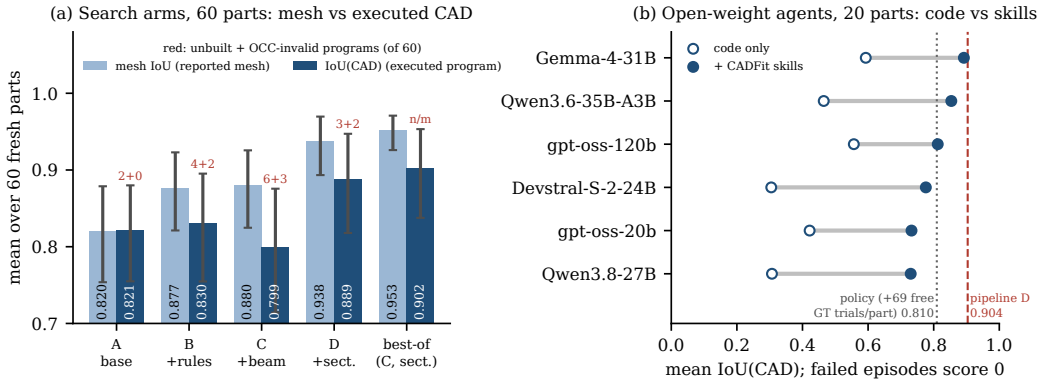


Figure 1: **(a)** Search arms on 60 fresh parts. Light bars score the mesh the search reports; dark bars score the solid built by executing the final program. Error bars: 95% bootstrap intervals of the mean over parts. Red: unbuilt + OCC-invalid programs; n/m = validity not measured for the best-of control. **(b)** Six open-weight agents on 20 of those parts, writing CadQuery alone (open) or with CADFit stages as tools (filled); failed episodes score 0. Vertical lines: the strongest pipeline arm on the same parts and two scripted policies over the same tools; the unrestricted one is labelled with its free ground-truth trials.

Q3 Do the optimizer's own stages make open-weight LLMs competent CAD agents, and does the LLM add anything over a script that uses the same stages?

Our answers, summarised in Figure 1:

1. **Execution changes the ranking of search extensions** (Section 4). The gap between mesh IoU and IoU_{CAD} grows from the base pipeline (-0.001) to beam search (0.081), almost entirely through programs that do not build. Gains in IoU_{CAD} are measurably smaller than gains in mesh IoU for the rule package and for the full ladder of extensions. We also document measurement defects in the stock objective and judge, including an alpha-wrapped objective, cavity filling and self-intersecting lofts, that change individual scores (Section 3).
2. **Whole-part section proposals survive execution** (Section 4). Adding a mid-plane section extrusion per axis to the candidate pool is the largest observed IoU_{CAD} increment among our extensions (+0.089, 95% interval [+0.008, +0.174]). The gain is concentrated on benchE; over the corrected base pipeline the overall IoU_{CAD} gain is not established.
3. **Optimizer stages are effective tools for open-weight agents** (Section 5). With the stages as tools, all six models improve over writing CadQuery alone, by 0.256 to 0.471 IoU_{CAD}, and every Bonferroni-corrected interval excludes zero. The best agent's mean is -0.012 relative to the strongest pipeline arm on the same parts (interval $[-0.050, +0.041]$). A scripted policy with the same tools and budget is not shown to be worse than any agent.

The evaluation protocol, the extensions, the agent harness and every script that regenerates our tables are released with the paper.

2 Related work

Mesh- and point-cloud-to-CAD. Learned methods map geometry to construction sequences: DeepCAD (Wu et al., 2021) and Point2Cyl (Uy et al., 2022) predict sketch-and-extrude structure, and CAD-Recode (Rukhovich et al., 2025) decodes point clouds into executable CadQuery code with a small LLM. CADFit (Nehme et al., 2026) instead optimizes: it slices the mesh into sketches, fits extrusion, revolve, fillet and chamfer candidates, and greedily composes them to maximize volumetric IoU, which lowers the invalid ratio on complex parts. Datasets include ABC (Koch et al., 2019), Fusion 360 Gallery (Willis et al., 2021) and DeepCAD; CADBench (Doris et al., 2026) unifies them with face-count tiers and metrics for fidelity, executability and compactness. We do not

propose a new reconstructor. We audit what an optimizer's reported score measures once its final program is replayed, and which search extensions survive that replay.

LLM agents with CAD tools. Tool-using language models (Yao et al., 2023; Schick et al., 2023) and agent–computer interfaces (Yang et al., 2024) motivate CAD agents. CAD-Assistant (Mallis et al., 2025) plans FreeCAD actions with CAD-specific tools such as a sketch parameterizer and a cross-section generator. CADFather (Savrasov et al., 2026) coordinates learned and algorithmic proposal tools with a vision-language assistant, executes the proposed programs and keeps the best valid result. Our tools are the stages of one optimizer, and our controls are chosen to separate the tools from the model: the same agents without the tools, with only one tool, and two scripted policies that use the tools without an LLM.

Table 1 summarises the comparison.

Table 1: Positioning. "Executes final program in eval": the reported score is computed on the solid produced by running the returned program. "Non-LLM policy control": a scripted policy over the same tools and budget.

Work	Input	Geometric tools / proposals	Executes final program in eval	Non-LLM policy control
CADFit	mesh	built-in pipeline stages	candidates executed; search scored on meshes	–
CAD-Recode	point cloud	none (learned decoder)	yes	–
CAD-Assistant	multimodal queries	FreeCAD API + CAD tools	yes (interpreter)	no
CADFather	mesh	learned + algorithmic tools	yes	no
This work	mesh	CADFit stages as tools	yes (sandboxed replay)	yes

3 SETTING AND MEASUREMENT

Task and optimizer. Given a watertight triangle mesh, the goal is a CadQuery (CadQuery contributors, 2024) program whose solid matches it. All geometry is normalized so the target is centred and its largest extent spans $[-1, 1]$. CADFit (Nehme et al., 2026) slices the mesh into planar sections, fits lines and arcs to each loop, proposes extrusion candidates from a Chamfer-distance sweep over translations and revolve candidates from an axis analysis, and composes candidates greedily with union and cut. Backward pruning and residual refinement follow, and the final composition is emitted as one CadQuery program.

Measurement defects. While extending CADFit on development parts we found defects in how compositions were scored (Table 2). The stock objective alpha-wraps (Portaneri et al., 2022) the concatenated candidate meshes before computing IoU, which fills concave detail: on one smoke-test part the search believed 0.954 where an exact judge measured 0.982. A body-wise union filled internal cavities. Self-intersecting lofts passed watertightness and kernel validity checks while cancelling their own volume. With our exact-union objective (manifold booleans of cached candidate solids), the scored mesh is never produced by running the composed program. Table 2 lists each with the case where we observed it. These are case studies on development data; all are fixed before the fresh evaluation, and aggregate gains are not attributed to individual fixes.

Evaluation protocol. Mesh IoU compares the method's output mesh with the target: exact volumes from manifold booleans (Lambert & Manifold contributors, 2024) when the output is a closed, consistently wound volume, otherwise a seeded Monte Carlo estimate in which inside-tests use generalized winding numbers (Jacobson et al., 2013). $\mathrm{IoU}_{\mathrm{CAD}}$ applies the same judge to the solid obtained by running the final program in a fresh, sandboxed process (no network; writes confined to its folder). A program that does not build scores 0. OpenCascade validity of the shape and every solid is recorded; $\mathrm{IoU}_{\mathrm{CAD}}$ keeps the measured score of a built but invalid solid, and the validity-gated $\mathrm{IoU}_{\mathrm{CAD}}^{\mathrm{valid}}$ is 0 unless the program builds and is valid. Chamfer distance (CD) is averaged over parts with an output, with its denominator stated.

Fresh cohort and statistics. CADBench's benchA and benchE splits stratify ABC-derived parts by B-rep face count; we use the hard tier. The fresh cohort is a seeded draw of 30 benchA-hard and 30 benchE-hard parts taken after, and disjoint from, all test, development and smoke sets used earlier. All arms run once per part from one code version. Differences are paired by part; intervals

Table 2: Measurement defects found on development, smoke and earlier test parts (not the fresh cohort), each with the case where it was observed. All were fixed before the fresh evaluation. Records: `results/defect_cases.json`.

defect	code path / fix	case	before	after
stock greedy objective alpha-wraps the concatenated candidate STLs before taking IoU	greedy_search.py (stock composition scoring); fix –exact-union	smoke 00053793	search-internal IoU 0.954	strict judge 0.982
body-wise union fills internal cavities (negative-volume shells unioned)	iou._bodies_union_manifold	00344388 (beam); 00014841 (rules)	0.9758; 0.9102	0.9862; 0.9475
under –exact-union the scored mesh comes from manifold booleans; the final CadQuery program is not executed when scored	bench/score.py (pre –cad); fix bench/cad_replay.py + score.py –cad	p3test2 beam 00706786; 00409618; 00014841	00706786 scored 0.980 as valid	1/4 solids OCC-invalid; 00409618 2/33 invalid; 00014841 beam program does not build (Null TopoDS_Shape)
self-intersecting lofts pass watertight/winding/isValid checks but cancel volume	analyze_sweep_loft.py candidates; fix: reject alpha-wrap/mesh volume ratio > 1.12	one loft candidate; 00399282	greedy believed 0.998	true score 0.776; 15 of 85 loft/sweep candidates rejected on 00399282
post-greedy re-measure alpha-wraps any multi-body mesh, filling gaps, triggering the single-candidate fallback	run_pipeline re-measure; fix per-body exact union	heatsink 00965166	0.978 composition replaced by 0.878 single extrusion	composition kept
trimesh API change makes the discrete() fallback throw, silently dropping slices	extract_sketches.py, extract_planar_contours.py; fix –fix-slice-fallback	smoke set	7 slices dropped on 4 smoke parts	0 dropped
pipeline summary counts a part as successful when greedy_search crashed	run_pipeline.py summary	smoke run with a mid-run edit	reported 5/5 successful	3 parts had crashed; only the strict judge catches it

Table 3: Search arms on 60 fresh parts. IoU_{CAD}: IoU of the solid built by executing the final program (0 if it does not build); $\text{IoU}_{\text{CAD}}^{\text{valid}}$: additionally 0 unless the shape is OpenCascade-valid. CD over parts with an output (n). Unbuilt + invalid: programs that do not build + built but invalid. n/m: not measured.

	mesh IoU	IoU_{CAD}	$\text{IoU}_{\text{CAD}}^{\text{valid}}$	gap	CD (n)	unbuilt + inv.	IoU_{CAD} benchA	IoU_{CAD} benchE
A base	0.820	0.821	0.821	−0.001	0.0155 (58)	2 + 0	0.866	0.777
B + rules	0.877	0.830	0.799	0.046	0.0130 (59)	4 + 2	0.837	0.824
C + beam	0.880	0.799	0.753	0.081	0.0129 (59)	6 + 3	0.775	0.824
D + sections	0.938	0.889	0.856	0.049	0.0105 (59)	3 + 2	0.859	0.918
best-of (C, sections)	0.953	0.902	n/m			n/m		
sections only	0.842							

paired difference	Δ mesh IoU	Δ IoU_{CAD}	Δ $\text{IoU}_{\text{CAD}}^{\text{valid}}$	ΔIoU_{CAD} − Δmesh
B − A (rules)	+0.056 [+0.027, +0.098]	+0.009 [−0.058, +0.070]	−0.022 [−0.100, +0.050]	−0.047 [−0.105, −0.003]
C − B (beam)	+0.003 [+0.0004473, +0.007]	−0.031 [−0.094, +0.029]	−0.047 [−0.122, +0.016]	−0.034 [−0.097, +0.026]
D − C (sections)	+0.058 [+0.026, +0.101]	+0.089 [+0.008, +0.174]	+0.103 [+0.029, +0.182]	+0.032 [−0.044, +0.109]
D − A (all)	+0.117 [+0.070, +0.175]	+0.067 [−0.012, +0.144]	+0.035 [−0.057, +0.121]	−0.050 [−0.109, −0.003]

are percentile bootstrap over parts (Efron & Tibshirani, 1993) (10,000 resamples, seed 0) and reflect part sampling, not run-to-run randomness. Search-arm comparisons are exploratory with nominal 95% intervals. For the agent study, comparisons across the six models use Bonferroni-corrected 99.17% intervals ($\alpha = 0.05/6$). Win/loss counts use a tie threshold of 10^{-9}.

4 What survives execution: search extensions on a fresh cohort

Arms. All four arms share one code version, seed 0 and one refinement iteration. **A** is CADFit with the measurement fixes of Section 3. **B** adds a rule package: a persistent CadQuery interpreter (byte-identical outputs, about 90% less per-call startup), exact-union scoring, a cut phase that subtracts candidate solids while IoU improves, event slices between consecutive heights of axis-normal planar faces, extrusion directions from clustered face normals, and a subsampled translation sweep. **C** adds beam search (width 4) over compositions. **D** adds whole-part section proposals: for each axis, the target's mid-bounding-box cross-section, simplified and extruded through the full extent of the part, enters the candidate pool. This recipe began as a control an external reviewer designed after seeing agent results on test parts; we froze it on development parts before drawing the fresh cohort. As a further control, *best-of* takes, per part, whichever of C's model and the best single section extrusion has the higher IoU against the target mesh.

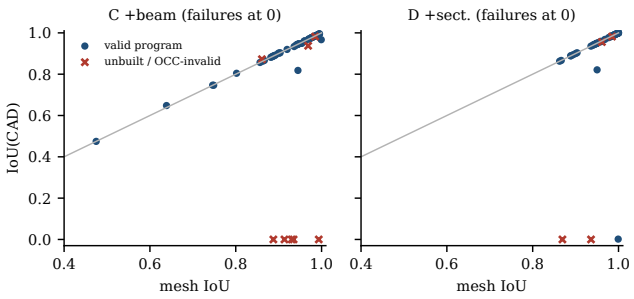


Figure 2: Per-part mesh IoU against IoU_{CAD} for arms C and D (60 parts). Crosses: programs that do not build (plotted at 0) or build but fail OpenCascade validity. Points on the diagonal reproduce their reported mesh.

Mesh IoU overstates the executed program, through programs that do not build (Q1). The base arm's executed programs reproduce its meshes: the mean gap between mesh IoU and IoU_{CAD} is -0.001 (Table 3). The gap rises to 0.046 with the rule package and 0.081 with beam search, then falls to 0.049 with section proposals. In arm C, 6 programs do not build and 3 build but fail OpenCascade validity. Averaged over all 60 parts, the unbuilt programs contribute 0.078 of the 0.081 gap and built programs 0.003. Built programs usually reproduce their meshes, though not always: the largest built-program discrepancy in arm C is 0.127 (Figure 2).

Gains shrink when programs are executed (Q2). Each extension raises mean mesh IoU, and every mesh-IoU interval in Table 3 excludes zero. The last column tests whether the executed-program gain is smaller than the mesh gain on the same parts. It is for the rule package (-0.047, $[-0.105, -0.003]$) and for the full ladder D $-$ A (-0.050, $[-0.109, -0.003]$); for beam search alone the test is inconclusive ($[-0.097, +0.026]$). Win/loss counts show how a few failures dominate a mean: under $\text{IoU}_{\text{CAD}}^{\text{valid}}$, the rule package improves 39 parts and worsens 7, yet its mean change is -0.022, because the parts it worsens lose their whole score.

Whole-part section proposals survive execution, mainly on benchE (Q2). Section proposals are the largest observed IoU_{CAD} increment among the extensions: D $-$ C is +0.089 [+0.008, +0.174], and D has the highest IoU_{CAD} of the four arms. A single best-axis section extrusion with no search reaches a mean mesh IoU of 0.842, above the base arm's 0.820. Folding the proposals into the search is not distinguishable from picking the better of C and the best section per part (D $-$ best-of on IoU_{CAD}: -0.014 $[-0.080, +0.053]$). Over the base arm, the overall IoU_{CAD} gain of D is +0.067 $[-0.012, +0.144]$, and it comes from benchE (+0.141 [+0.025, +0.261]) rather than benchA (-0.006 $[-0.112, +0.073]$). We do not test a split interaction, and with one run per part we do not attribute gains to individual rules.

5 Optimizer stages as tools for open-weight agents

Having identified which geometric proposals survive execution, we test what LLM agents gain from access to the optimizer's operations, and whether a script with the same operations does as well.

Harness. An agent receives no image or mesh. It learns about the target only through tools, over an OpenAI-compatible tool-calling loop served by vLLM (Kwon et al., 2023) on one GPU per model. Every toolset includes `describe_target` (size, volume, flat faces), `section` (exact cross-section polygons of the target and the current model on an axis-aligned plane), `evaluate` (IoU against the target and where material is missing or extra), `add_code` (union or cut of a solid built by agent-written CadQuery), `remove_step`, `list_steps` and `finish`. The **code** toolset is exactly this. The **skills** toolset adds five CADFit stages: `detect_directions`; `extract_sketch`, which slices the target at a plane, nests loops into profiles and fits lines and arcs; `fit_extrude` (translation sweep, then extents); `fit_revolve` (axis analysis); and `section_extrude` (the whole-part proposal of Section 4). Each stage's solids become can-

Table 4: Agents on 20 fresh parts, mean IoU_{CAD} (failed episodes score 0). Brackets: Bonferroni-corrected 99.17% paired bootstrap intervals over parts ($\alpha = 0.05/6$). W–L: parts where skills scored higher–lower than code. Pipeline D on the same parts: 0.904 ($\text{IoU}_{\text{CAD}}{}^{\text{valid}}$ 0.904); unrestricted policy 0.810; budget-matched policy **PENDING**.

model	code	skills	skills − code	W–L	$\text{IoU}_{\text{CAD}}{}^{\text{valid}}$ skills	skills − pipeline D	skills − matched policy
Gemma-4-31B	0.593	0.892	+0.299 [+0.138, +0.481]	17–3	0.892	−0.012 [−0.050, +0.041]	**PENDING PENDING**
Qwen3.6-35B-A3B	0.465	0.854	+0.389 [+0.206, +0.567]	17–3	0.854	−0.050 [−0.122, +0.026]	**PENDING PENDING**
gpt-oss-120b	0.557	0.812	+0.256 [+0.112, +0.417]	18–2	0.769	−0.091 [−0.221, +0.010]	**PENDING PENDING**
Devstral-S-2-24B	0.305	0.776	+0.471 [+0.319, +0.613]	19–1	0.737	−0.128 [−0.260, −0.001]	**PENDING PENDING**
gpt-oss-20b	0.422	0.732	+0.311 [+0.104, +0.519]	15–5	0.732	−0.171 [−0.340, −0.047]	**PENDING PENDING**
Qwen3.8-27B	0.307	0.730	+0.423 [+0.178, +0.663]	15–3	0.730	−0.174 [−0.369, −0.031]	**PENDING PENDING**

didates, scored against the target when created, which the agent can list, inspect and add. The **code+section** toolset adds only `section_extrude` and the candidate tools. The system prompts differ only in the tool paragraph and one strategy phrase. Episodes start from an empty model, allow 40 tool calls, 30 minutes and 16,384 generated tokens per turn, and return the best model seen, as judged by IoU against the target; a failed episode scores 0. Target-score feedback is legitimate here, since the target mesh is the method's input. Agent-written code runs in a sandbox. Three implementation details make the skills differ from the frozen pipeline arms: even-odd loop nesting (the pipeline treated a material island inside a hole as a second hole), a 10^{-6} minimum profile area (the pipeline's 10^{-3} dropped all 67 profiles of one slice), and a fix to how revolve profiles are split at the axis (a stepped shaft went from 0.408 to 0.994 IoU).

Models and parts. Six open-weight models, each on one GPU, with model-card sampling and reasoning enabled where the model supports it: Gemma-4-31B (Google DeepMind, 2026), Qwen3.6-35B-A3B and Qwen3.8-27B (Yang et al., 2025), gpt-oss-120b and gpt-oss-20b (OpenAI et al., 2025), and Devstral-Small-2-24B (Mistral AI, 2025). We use 20 parts (12 benchA, 8 benchE) drawn by seed from the fresh cohort before any episode ran, with one episode per model, toolset and part.

Scripted policies. The *unrestricted* policy calls `section_extrude` on each axis, extracts midplane sketches along the detected directions, fits the three largest profiles per plane, and assembles greedily by best improvement. Its assembly scores every candidate union and cut against the target without spending calls: 69 free trials per part on average. The *budget-matched* policy charges every such trial as one of its 40 calls, keeps the first improving trial (candidates in IoU order, adds before cuts), and reserves 20 calls for assembly. It used **PENDING** calls per part, **PENDING** of them trials. An agent pays at least as much per trial, since adding a candidate reports the new IoU and undoing it costs another call.

The stages make every model a better CAD agent (Q3). With the stages as tools, all six models improve over writing CadQuery alone (Table 4), by 0.256 to 0.471 IoU_{CAD}, and every Bonferroni-corrected interval excludes zero. The largest gains go to the weakest code-writers: Devstral rises from 0.305 to 0.776 and Qwen3.8 from 0.307 to 0.730. Validity gating lowers a skills score by at most 0.044. Gemma-4-31B with skills reaches 0.892, an observed −0.012 relative to pipeline D on the same parts ([−0.050, +0.041]). Its three runs over the 20 parts score **PENDING**, **PENDING** and **PENDING** (mean **PENDING**, range **PENDING**–**PENDING**); the first is the declared primary result, and the range describes three runs, not an interval.

Ablation: code plus section extrusion only. Given only `section_extrude` beyond the code tools, Gemma scores **PENDING**: **PENDING PENDING** over code alone, and the full skills toolset adds **PENDING PENDING** on top (nominal 95% intervals, Gemma only).

The LLM's added value over a script is not established. Against the unrestricted policy (0.810), Gemma's difference is +0.083 with Bonferroni interval [−0.0001087, +0.234], and every other model's interval also includes zero. Against the budget-matched policy (**PENDING**), the last column of Table 4 gives each model's difference. An LLM benefit would require a lower bound above zero against this policy.

How the agents use the stages. Per episode, agents call `extract_sketch` 1.8–4.9 times and `fit_extrude` 1.6–6.8 times, but `fit_revolve` at most 0.3 times; these parts rarely call for revolves (Appendix B). The dominant failure is not geometric. At 16,384 tokens per turn, 16 of 20 Qwen3.8 code episodes terminated on a turn without a tool call. In all 16 of them, the preceding reply had hit the token limit (the harness then asks the model to think briefly and call a tool), and the final reply was 32.5–39.4 thousand characters of reasoning with no content (Appendix C).

6 Discussion and conclusion

Executing the returned program changes what a mesh-to-CAD search appears to achieve. Stronger search raises reported mesh IoU on every step we measured, but it also produces programs that do not build, and for the rule package and the full ladder the executed-program gain is measurably smaller than the mesh gain. Whole-part section proposals are the extension whose gain holds up best under execution. The same operations, offered as tools, raise every one of six open-weight models well above what it achieves by writing CadQuery alone; the best agent's mean differs from the pipeline's by -0.012 on the same parts. Whether the LLM adds anything over a script that uses the same tools is the question our data leave open. Answering it is the natural next experiment, because CAD-agent comparisons that omit such a policy cannot separate the tools from the model.

Limitations. (i) Each search arm and each agent condition ran once per part, except three Gemma runs; our intervals capture part sampling, not run-to-run variation. (ii) The agent study uses 20 parts, which leaves intervals wide; the policy comparison in particular is underpowered. (iii) The cohort is hard-tier benchA/benchE, dominated by axis-aligned 2.5D parts: agents rarely called `fit_revolve`, so revolve-heavy parts are untested. (iv) We audit one optimizer family. The mesh–program gap we report is a property of these searches, and other reconstructors need their own replay audit. (v) The agent's revolve tool includes a profile-split fix that the frozen pipeline arms lack, and the section-proposal recipe originated from a control designed after seeing agent results on test parts, although it was frozen before the fresh cohort was drawn. (vi) All conditions select answers with IoU against the target mesh; this is the task's input, but results do not transfer to settings without that feedback.

Reproducibility statement

Every number in the paper is a macro generated from result files: `bench/paper_stats.py` computes `results/*.json` from the per-part scores (`bench/runs/fresh_scores.json`, `fresh_section_control.json`, `agentsk_*scores.json`), and `bench/paper_numbers.py` writes `paper/numbers.tex`; `paper/figures/make_figures.py` draws the figures. Per-part scores come from `bench/agentsk_score.py` and `bench/score.py -cad`. Model identifiers, serving flags and sampling settings are in `bench/agentsk_models.py`; the cohort manifest is `bench/cadbench/index.json` with the fresh file list, and the agent sample is `study_parts` with seed 1. The pipeline arms ran from frozen per-arm code snapshots, which are released; the agent study and all analysis code correspond to the repository tag `paper-v1`. Every under-determined design decision of the agent study is recorded in a decision ledger released with the code.

References

CadQuery contributors. CadQuery: A python parametric CAD scripting framework based on OCCT. https://github.com/CadQuery/cadquery, 2024.

Anna C. Doris, Jacob Thomas Sony, Ghadi Nehme, Era Syla, Amin Heyrani Nobari, and Faez Ahmed. Cadbench: A multimodal benchmark for ai-assisted cad program generation, 2026. URL https://arxiv.org/abs/2605.10873.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

Google DeepMind. Gemma 4 model card (google/gemma-4-31B-it). https://huggingface.co/google/gemma-4-31B-it, 2026.

Alec Jacobson, Ladislav Kavan, and Olga Sorkine-Hornung. Robust inside-outside segmentation using generalized winding numbers. *ACM Transactions on Graphics*, 32(4), 2013. doi: 10.1145/2461912.2461916.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning, 2019. URL https://arxiv.org/abs/1812.06216.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Emmett Lambert and Manifold contributors. Manifold: geometry library for topological robustness. https://github.com/elalish/manifold, 2024.

Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: Tool-augmented vllms as generic cad task solvers, 2025. URL https://arxiv.org/abs/2412.13810.

Mistral AI. Devstral small 2 (mistralai/Devstral-Small-2-24B-Instruct-2512). https://huggingface.co/mistralai/Devstral-Small-2-24B-Instruct-2512, 2025.

Ghadi Nehme, Eamon Whalen, and Faez Ahmed. Cadfit: Precise mesh-to-cad program generation with hybrid optimization, 2026. URL https://arxiv.org/abs/2605.01171.

OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Cédric Portaneri, Mael Rouxel-Labbé, Michael Hemmer, David Cohen-Steiner, and Pierre Alliez. Alpha wrapping with an offset. *ACM Transactions on Graphics*, 41(4), 2022. doi: 10.1145/3528223.3530152.

Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds, 2025. URL https://arxiv.org/abs/2412.14042.

Gennadiy Savrasov, Maksim Elistratov, Nikita Gavrilov, Albert Garifullin, Oleg Pavlov, Soslan Kabisov, Vladimir Frolov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadfather: Autonomous cad reconstruction through coordinated tool use, 2026. URL https://arxiv.org/abs/2610.09127.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Mikaela Angelina Uy, Yen yu Chang, Minhyuk Sung, Purvi Goel, Joseph Lambourne, Tolga Birdal, and Leonidas Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders, 2022. URL https://arxiv.org/abs/2112.09329.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences, 2021. URL https://arxiv.org/abs/2010.02392.

Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models, 2021. URL https://arxiv.org/abs/2105.09492.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

A AGENT HARNESS DETAILS

Sandbox. Agent-written code and the replay of every final program run in a sandbox with no network and writes confined to the episode folder: bubblewrap where available; otherwise an unprivileged user namespace in which every mount is remounted read-only and verified (the sandbox refuses to run if any mount stays writable), temporary directories are replaced by empty file systems, the episode folder is bound back read-write, and all capabilities are dropped with no-new-privileges set.

Context. The last six tool results are kept in full; older results, older assistant text and older code arguments are truncated to 300 characters in what is sent back. Tool results are capped at 8,000 characters while remaining valid JSON. On a context-length error the kept window halves.

Models.

model	temperature	request extras and vLLM parsers
Gemma-4-31B (`google/gemma-4-31B-it`)	1.0	`{"top_p": 0.95, "top_k": 64, "chat_template_kwargs": {"enable_thinking": true}}; -reasoning-parser gemma4 -tool-call-parser gemma4`
Qwen3.6-35B-A3B (`Qwen/Qwen3.6-35B-A3B-FP8`)	1.0	`{"top_p": 0.95, "top_k": 20, "min_p": 0.0, "presence_penalty": 0.0, "chat_template_kwargs": {"enable_thinking": true}}; -reasoning-parser qwen3 -tool-call-parser qwen3_coder -language-model-only`
gpt-oss-120b (`openai/gpt-oss-120b`)	1.0	`{"top_p": 1.0}; -reasoning-parser openai_gptoss -tool-call-parser openai`
Devstral-S-2-24B (`mistralai/Devstral-Small-2-24B-Instruct-2512`)	0.15	`{}; -tool-call-parser mistral -tokenizer-mode mistral -config-format mistral -load-format mistral`
gpt-oss-20b (`openai/gpt-oss-20b`)	1.0	`{"top_p": 1.0}; -reasoning-parser openai_gptoss -tool-call-parser openai`
Qwen3.8-27B (`Qwen/Qwen3.8-27B-FP8`)	1.0	`{"top_p": 0.95, "top_k": 20, "min_p": 0.0, "presence_penalty": 0.0, "chat_template_kwargs": {"enable_thinking": true}}; -reasoning-parser qwen3 -tool-call-parser qwen3_xml -language-model-only`

Prompts. The three system prompts differ only in the tool paragraph and the strategy phrase.

System prompt, `code` toolset.

```
You are a CAD engineer. Your job: build a solid CAD model that matches a target 3D part as
    closely as possible, measured by volumetric IoU (intersection over union) against the
    target.

You cannot see the target. You learn about it only through tools:
- describe_target: size, volume, and the flat faces (directions and plane positions).
- section: exact cross-section polygons of the target (and of your model) in any
    axis-aligned plane.
- add_code: append a solid built by your CadQuery code to the model as a union ('add') or a
    difference ('cut').
- evaluate: current IoU and where material is missing or extra.
- remove_step, list_steps, finish.

Coordinates: everything is in the normalized frame (the target is centered and scaled so
    its largest extent spans [-1, 1]). Your CadQuery code (cq and math are imported) must
    assign the final solid to `result`, e.g.
    result = cq.Workplane('XY').box(1.2, 0.8, 0.1).translate((0, 0, 0.35))
    result = cq.Workplane('XY', origin=(0, 0, -0.2)).circle(0.3).extrude(0.4)
    result =
    cq.Workplane('XZ').polyline([(0,0),(0.5,0),(0.5,0.2),(0,0.2)]).close().revolve(360,
    (0,0,0), (0,1,0))

Good strategy: describe the target, read sections to get exact profile shapes, build the
    main body first, then add features and cut holes/pockets, checking evaluate after
    changes and removing steps that lower IoU. The best model seen at any point is kept, so
    experimenting is safe. Work efficiently: you have a limited number of tool calls. Call
    finish when you cannot improve further.
```

System prompt, `codesec` toolset.

```
You are a CAD engineer. Your job: build a solid CAD model that matches a target 3D part as
    closely as possible, measured by volumetric IoU (intersection over union) against the
    target.

You cannot see the target. You learn about it only through tools:
- describe_target: size, volume, and the flat faces (directions and plane positions).
- section: exact cross-section polygons of the target (and of your model) in any
    axis-aligned plane.
- section_extrude: a CADFit stage that extrudes the target's whole cross-section at a plane
    through the full extent of the part. Each solid it makes becomes a scored candidate:
    list_candidates / show_candidate show them; precision near 1 means the candidate lies
    inside the target.
- add_candidate / add_code: append a solid to the model as a union ('add') or a difference
    ('cut').
- evaluate: current IoU and where material is missing or extra.
- remove_step, list_steps, finish.

Coordinates: everything is in the normalized frame (the target is centered and scaled so
    its largest extent spans [-1, 1]). Your CadQuery code (cq and math are imported) must
    assign the final solid to `result`, e.g.
    result = cq.Workplane('XY').box(1.2, 0.8, 0.1).translate((0, 0, 0.35))
    result = cq.Workplane('XY', origin=(0, 0, -0.2)).circle(0.3).extrude(0.4)
```

```
    result =
    cq.Workplane('XZ').polyline([(0,0),(0.5,0),(0.5,0.2),(0,0.2)]).close().revolve(360,
    (0,0,0), (0,1,0))

Good strategy: describe the target, read sections to get exact profile shapes and extrude
    whole cross-sections, build the main body first, then add features and cut
    holes/pockets, checking evaluate after changes and removing steps that lower IoU. The
    best model seen at any point is kept, so experimenting is safe. Work efficiently: you
    have a limited number of tool calls. Call finish when you cannot improve further.
```

System prompt, `skills` toolset.

```
You are a CAD engineer. Your job: build a solid CAD model that matches a target 3D part as
    closely as possible, measured by volumetric IoU (intersection over union) against the
    target.

You cannot see the target. You learn about it only through tools:
- describe_target: size, volume, and the flat faces (directions and plane positions).
- section: exact cross-section polygons of the target (and of your model) in any
    axis-aligned plane.
- CADFit skills, the stages of a CAD reconstruction pipeline, which you run yourself:
    detect_directions (likely extrusion directions), extract_sketch (slice the target at a
    plane into sketch profiles fitted with lines and arcs), fit_extrude / fit_revolve (turn
    a sketch into extrusion / revolve solids that match the target), section_extrude
    (extrude a whole cross-section through the part). Each solid they make becomes a scored
    candidate: list_candidates / show_candidate show them; precision near 1 means the
    candidate lies inside the target.
- add_candidate / add_code: append a solid to the model as a union ('add') or a difference
    ('cut').
- evaluate: current IoU and where material is missing or extra.
- remove_step, list_steps, finish.

Coordinates: everything is in the normalized frame (the target is centered and scaled so
    its largest extent spans [-1, 1]). Your CadQuery code (cq and math are imported) must
    assign the final solid to `result`, e.g.
    result = cq.Workplane('XY').box(1.2, 0.8, 0.1).translate((0, 0, 0.35))
    result = cq.Workplane('XY', origin=(0, 0, -0.2)).circle(0.3).extrude(0.4)
    result =
    cq.Workplane('XZ').polyline([(0,0),(0.5,0),(0.5,0.2),(0,0.2)]).close().revolve(360,
    (0,0,0), (0,1,0))

Good strategy: describe the target, extract sketches on its main planes and fit them into
    candidates, build the main body first, then add features and cut holes/pockets,
    checking evaluate after changes and removing steps that lower IoU. The best model seen
    at any point is kept, so experimenting is safe. Work efficiently: you have a limited
    number of tool calls. Call finish when you cannot improve further.
```

B TOOL USE

Tool calls per episode in the skills toolset, over the 20 episodes of each model (`results/tool_usage.json`).

model	calls	extract_sketch	fit_extrude	fit_revolve	section_extrude	detect_directions	section	add_candidate	add_code	evaluate	remove_step	errors
Gemma-4-31B	27.9	3.1	2.7	0.1	1.1	0.4	6.0	2.9	3.5	1.9	2.6	17/559
Qwen3.6-35B-A3B	37.2	4.9	2.2	0.0	1.6	1.0	12.3	2.5	3.4	2.2	2.5	47/744
gpt-oss-120b	38.2	4.8	3.6	0.1	1.6	1.0	6.6	4.0	3.6	3.0	3.4	22/764
Devstral-S-2-24B	37.2	3.8	6.8	0.3	1.2	0.9	2.7	7.2	3.4	4.1	3.9	12/745
gpt-oss-20b	25.1	2.1	1.8	0.1	1.6	0.9	6.2	2.6	2.2	2.6	1.2	44/503
Qwen3.8-27B	22.2	1.8	1.6	0.0	0.3	1.0	7.9	1.3	3.1	1.3	1.8	27/445

C QWEN3.8 TOKEN-LIMIT FAILURES AND A POST-HOC SENSITIVITY RUN

The harness sends "Your reply hit the token limit before you called a tool. Think briefly, then call a tool now." only when a reply ends because it reached the 16,384-token limit; after two consecutive nudges without a tool call the episode stops. In all 16 Qwen3.8 code episodes that stopped this way, the last nudge was the token-limit message, and the final reply was 32.5–39.4 thousand characters

of reasoning with no content (`results/qwen38_traces.json`). After seeing these outcomes we reran Qwen3.8 on the same parts with 32,768 tokens per turn. It scored 0.783 with skills and 0.541 with code (versus 0.730 and 0.307 at 16,384). Because this run was decided after seeing results, it is excluded from Table 4; most of its episodes ended at the 30-minute limit instead.

D Refining pipeline output with an LLM agent

In an earlier study, Qwen3.8-27B (INT4) refined arm C's output on the 19 fresh parts whose C mesh IoU was below 0.90 (gate fixed before running), with access to C's candidate pool, 40 calls and 20 minutes, in two runs. Mean mesh IoU: pipeline 0.751; agent 0.797 and 0.807 (wins–losses against the pipeline at a ± 0.01 threshold: 2–3 and 4–3); a no-op agent that only re-exports the pipeline model 0.749; an equal-time scripted search over the same pool 0.756 (8–2). The mean gain comes from a few parts and does not repeat consistently. Per-part records were not retained, so no intervals or executed-program scores are available for this study.

E Per-part results

part	split	A mesh	A CAD	B mesh	B CAD	C mesh	C CAD	D mesh	D CAD
00038011	A	0.844	0.844	0.939	0.937	0.940	0.940	0.940	0.940
00046097	A	0.896	0.915	0.939	0.940	0.942	0.943	0.942	0.943
00053172	E	0.994	0.994	0.994	0.994	0.994	0.994	0.998	0.998
00090768	A	0.962	0.962	0.971	0.971	0.999	0.967	0.999	0.002
00095369	A	0.882	0.882	0.893	0.893	0.893	0.893	0.948	0.948
00101419	A	0.965	0.965	0.978	0.978	0.978	0.978	1.000	1.000
00140273	E	0.884	0.884	0.884	0.884	0.884	0.884	0.894	0.894
00153120	E	0.771	0.771	0.857	0.857	0.857	0.857	0.996	0.996
00154184	E	0.000	–†	0.980	0.981	0.984	0.984	0.984	0.984
00161800	A	0.985	0.985	0.985	0.985	0.985	0.985	0.985	0.985
00181393	A	0.863	0.863	0.863	0.863	0.863	0.863	0.863	0.863
00208290	E	0.989	0.989	0.989	0.989	0.989	0.989	0.992	0.992
00214289	A	0.997	0.997	0.989	0.989	0.989	0.989	0.989	0.989
00232213	E	0.837	0.837	0.869	0.000†	0.861	0.873*	0.869	0.000†
00246395	E	0.959	0.959	0.959	0.959	0.959	0.959	0.997	0.997
00247424	E	0.000	–†	0.000	–†	0.000	–†	0.000	–†
00249450	E	0.926	0.926	0.960	0.960	0.960	0.960	0.995	0.995
00256926	E	0.814	0.814	0.892	0.892	0.899	0.902	1.000	1.000
00266468	E	0.723	0.724	0.746	0.746	0.749	0.746	1.000	1.000
00273535	E	0.912	0.912	0.995	0.995	0.995	0.995	1.000	1.000
00299355	E	0.975	0.976	0.975	0.976	0.975	0.976	0.990	0.990
00305871	E	0.960	0.960	0.961	0.961	0.961	0.961	1.000	1.000
00312587	E	0.973	0.973	0.975	0.974	0.975	0.974	1.000	1.000
00317970	E	0.182	0.182	0.733	0.743	0.746	0.747	0.993	0.993
00354176	A	0.896	0.896	0.903	0.904	0.903	0.904	0.903	0.904
00365747	A	0.848	0.848	0.880	0.000†	0.888	0.000†	0.961	0.956*
00382121	A	0.920	0.920	0.933	0.933	0.936	0.934	0.936	0.936
00492401	A	0.975	0.986	0.986	0.986	0.987	0.988	0.987	0.987
00528181	A	0.860	0.876	0.915	0.916*	0.882	0.882	0.960	0.958
00562677	A	0.867	0.867	0.889	0.889	0.889	0.889	0.889	0.889
00589302	A	0.921	0.921	0.966	0.966	0.969	0.937*	0.949	0.949
00630878	A	0.848	0.849	0.942	0.944	0.944	0.944	0.944	0.944
00634093	A	0.931	0.931	0.941	0.942	0.946	0.946	0.946	0.946
00642725	A	0.908	0.908	0.931	0.000†	0.930	0.000†	0.936	0.000†
00657371	E	0.901	0.902	0.901	0.902	0.902	0.903	0.987	0.987
00678171	E	0.935	0.935	0.945	0.945	0.945	0.945	0.949	0.949
00700032	A	0.855	0.855	0.885	0.881	0.886	0.886	0.955	0.955
00709520	A	0.888	0.889	0.888	0.889	0.888	0.889	0.888	0.889
00729452	A	0.051	0.053	0.280	0.281	0.280	0.281	0.280	0.281
00730725	E	0.018	0.018	0.018	0.018	0.018	0.018	0.949	0.949
00771160	E	0.890	0.900	0.974	0.974	0.985	0.982*	0.985	0.982*
00771600	E	0.919	0.919	0.919	0.919	0.920	0.920	0.999	0.999
00780711	E	0.911	0.911	0.973	0.973	0.983	0.983	0.983	0.983
00791454	E	0.970	0.970	0.970	0.970	0.970	0.970	0.990	0.990
00795057	E	0.949	0.949	0.969	0.969	0.980	0.980	0.997	0.997
00814797	E	0.933	0.933	0.970	0.970	0.970	0.970	0.999	0.999
00834538	A	0.920	0.920	0.948	0.948	0.954	0.949	0.968	0.968
00846352	A	0.859	0.859	0.859	0.859	0.867	0.867	0.867	0.867
00862084	A	0.593	0.593	0.553	0.557	0.639	0.648	0.999	0.999
00862085	E	0.422	0.422	0.475	0.475	0.475	0.475	0.999	0.999
00862953	E	0.796	0.796	0.802	0.804	0.802	0.804	0.980	0.980
00883892	E	0.792	0.792	0.905	0.906	0.913	0.000†	0.900	0.900

part	split	A mesh	A CAD	B mesh	B CAD	C mesh	C CAD	D mesh	D CAD
00892314	A	0.916	0.916	0.933	0.940*	0.935	0.000†	0.959	0.959
00904252	A	0.977	0.977	0.977	0.977	0.977	0.977	0.977	0.977
00941601	A	0.923	0.923	0.943	0.943	0.947	0.947	0.949	0.949
00953658	E	0.973	0.973	0.976	0.976	0.978	0.978	0.982	0.982
00966049	A	0.815	0.815	0.945	0.818	0.945	0.818	0.950	0.821
00966713	E	0.994	0.994	0.994	0.994	0.994	0.994	0.998	0.998
00983172	A	0.877	0.877	0.993	0.985	0.993	0.000†	0.984	0.984
00992810	A	0.879	0.879	0.995	0.995	0.995	0.995	0.997	0.997

† program does not build; * builds but fails OpenCascade validity.

Table 6: Per-part $\mathrm{IoU}_{\mathrm{CAD}}$ of the agent study (failures 0).

part	Gemma-4-31B		Qwen3.6-35B-A3B		gpt-oss-120b		Devstral-S-2-24B		gpt-oss-20b		Qwen3.8-27B		policy	pipe D
	code	skills	code	skills	code	skills	code	skills	code	skills	code	skills		
00046097	0.389	0.832	0.000	0.739	0.702	0.770	0.274	0.765	0.844	0.739	0.000	0.785	0.776	0.943
00095369	0.955	0.951	0.945	0.872	0.866	0.953	0.086	0.838	0.873	0.872	0.945	0.986	0.899	0.948
00140273	0.251	0.894	0.439	0.884	0.446	0.894	0.308	0.894	0.245	0.647	0.000	0.000	0.894	0.894
00153120	0.976	0.996	0.410	0.996	0.976	0.996	0.227	0.971	0.579	0.996	0.749	0.996	0.996	0.996
00154184	0.230	0.967	0.168	0.936	0.268	0.218	0.211	0.218	0.151	0.195	0.000	0.971	0.810	0.984
00232213	0.121	0.279	0.399	0.304	0.228	0.276	0.126	0.458	0.243	0.000	0.000	0.000	0.403	0.000
00249450	0.980	0.979	0.980	0.995	0.942	0.995	0.327	0.881	0.651	0.995	0.978	0.980	0.995	0.995
00354176	0.021	0.929	0.221	0.929	0.313	0.930	0.221	0.929	0.281	0.929	0.566	0.929	0.000	0.904
00492401	0.647	0.932	0.231	0.934	0.602	0.847	0.343	0.883	0.292	0.848	0.000	0.910	0.871	0.987
00528181	0.221	0.889	0.472	0.765	0.196	0.889	0.252	0.706	0.164	0.765	0.000	0.732	0.784	0.958
00589302	0.747	0.948	0.950	0.974	0.950	0.948	0.652	0.669	0.950	0.948	0.000	0.824	0.948	0.949
00630878	0.252	0.951	0.000	0.658	0.255	0.747	0.181	0.676	0.265	0.615	0.000	0.835	0.746	0.944
00700032	0.961	0.962	0.608	0.962	0.960	0.962	0.326	0.962	0.614	0.962	0.975	0.963	0.962	0.955
00771600	0.944	0.999	0.618	0.999	0.618	0.621	0.713	0.919	0.000	0.999	0.000	0.985	0.999	0.999
00834538	0.816	0.969	0.319	0.968	0.816	0.968	0.183	0.966	0.816	0.968	0.813	0.968	0.968	0.968
00862084	0.933	0.999	0.157	0.999	0.489	0.905	0.161	0.798	0.124	0.993	0.926	0.975	0.999	0.999
00883892	0.668	0.773	0.590	0.669	0.676	0.789	0.590	0.572	0.590	0.793	0.169	0.000	0.793	0.900
00941601	0.289	0.898	0.332	0.740	0.030	0.873	0.489	0.731	0.338	0.000	0.000	0.916	0.740	0.949
00953658	0.628	0.929	0.622	0.929	0.623	0.929	0.415	0.893	0.415	0.622	0.000	0.844	0.929	0.982
00966049	0.830	0.764	0.836	0.823	0.176	0.735	0.011	0.790	0.000	0.763	0.018	0.000	0.679	0.821